LENSMETRICS

Smart Photo Quality & Structure Inspector

Computer Vision (CSE3010)

Evaluated Course Project

Submitted By:

Trisha

Registration Number: 24BAC10012

Stack: Python, OpenCV, NumPy

Testing: Pytest

Academic Year: 2026–27

Contents

1 Overview **1**

2 Motivation and Problem **1**

3 Goals **1**

4 Scope, Users, and Deliverables **2**
4.1 In Scope . . . 2
4.2 Out of Scope . . . 2
4.3 Who It Serves . . . 2
4.4 Deliverables per Run . . . 2

5 System Design **2**

6 Module Catalogue **3**

7 Methods and Parameters **3**
7.1 Face Marking . . . 3
7.2 Geometry and Contrast . . . 3
7.3 Exposure and Focus . . . 3
7.4 Structure Tracing . . . 3
7.5 Colour Balance . . . 3

8 Command Line and Failure Behaviour **4**

9 Implementation Notes **4**

10 Reference Run-Through **5**

11 Evidence **5**

12 Testing **8**
12.1 Automated Suite . . . 8
12.2 Manual Checks . . . 8

13 Difficulties and Fixes **8**

14 Takeaways, Limits, and Next Steps **9**
14.1 Takeaways . . . 9
14.2 Limits . . . 9
14.3 Next Steps . . . 9

15 References **9**

16 Closing **9**

1 Overview

Everyday photo folders grow faster than anyone curates them. Some shots are soft, some are washed out, some hide faces worth tagging — and judging each file by eye is slow and inconsistent. **LensMetrics** answers with one terminal command: hand it a photograph and it reports who is in it, how the exposure and focus hold up, how much structure the scene contains, and which colours dominate, keeping visual proof alongside the numbers.

The tool is written in Python on top of OpenCV. A single run loads the photo, marks faces with numbered labels, measures geometry and contrast, grades exposure and focus against tunable cutoffs, traces edges and counts contours, profiles the B/G/R balance, and writes four artefacts: an annotated photo, an edge map, a readable summary sheet, and a JSON record for tooling. Eight automated Pytest cases guard the pipeline.

On the reference shot (`input/image.jpeg`, 1200 × 1600 portrait), LensMetrics marked **2 faces**, graded the frame **Well-exposed** (luma 123.80) and **In-focus** (score 122.02), traced **56,365 edge pixels (2.94%)** grouped into **859 contours**, and found green the dominant channel.

2 Motivation and Problem

Manually triaging photos means opening files one by one and guessing at brightness, sharpness, and content. That workflow has three recurring pains:

- **Subjective grading:** two people rarely agree whether a shot is "a bit dark" or "fine" without numbers behind the words.
- **Scattered checks:** face counts live in one script, histograms in another, edge previews nowhere — nothing ties them into one pass.
- **No receipts:** even when a check runs, there is rarely a saved visual plus a machine-readable log to show for it.

LensMetrics collapses those checks into one deterministic run: every number it prints is also drawn onto an image or written to a report file, so the result can be re-examined without re-running anything.

3 Goals

1. **One-command inspection:** `python main.py -input <photo> -outdir output` must be enough for a full pass, with sensible defaults for every threshold.
2. **Grades, not just numbers:** raw luma and Laplacian variance are translated into plain labels (Under/Well/Over-exposed; In-focus vs Soft-focus) so beginners can act on them.
3. **Structure census:** beyond a bare edge image, count contours and report edge density so "busy" vs "flat" scenes are comparable.
4. **Colour snapshot:** per-channel means plus a dominant-channel pick characterise white balance at a glance.
5. **Dual reporting:** every run emits both `summary.txt` (human) and `summary.json` (machine) describing the same outcome.
6. **Guarded testing:** eight Pytest cases cover geometry, clarity, structure, palette, annotation, reporting, and input validation.

4 Scope, Users, and Deliverables

4.1 In Scope

Single-photo inspection through a CLI: file-type gate, OpenCV loading, face marking, geometry/contrast readout, exposure and focus grading, Canny tracing with contour counting, colour profiling, and artefact writing. Tunable flags expose the Canny band and the focus cutoff without touching code.

4.2 Out of Scope

No batch folders, no GUI, no learned detectors, no editing or enhancement — the tool observes and reports; it never modifies the source photo.

4.3 Who It Serves

- **Photo beginners** auditing which of their shots are actually sharp.
- **Vision students** seeing grayscale conversion, smoothing, cascades, Canny, Laplacians, and contours cooperate in one codebase.
- **Builders** wanting a cheap pre-check before feeding images to heavier models.

4.4 Deliverables per Run

File	Content
`annotated.jpg`	Source photo with numbered face boxes.
`edgemap.jpg`	Canny structure map of the scene.
`summary.txt`	Readable sheet: faces, geometry, clarity, structure, colour.
`summary.json`	Same record as structured data, including box coordinates.

5 System Design

The pipeline is linear and stateless: each stage consumes plain arrays or dataclasses and the CLI module is the only coordinator.

`CLI` → `io_handler` → `face_inspection` ‖ `photo_stats` ‖ `clarity` ‖ `structure` ‖ `palette` → `visualizer` + `reporter`

1. **Intake** (`cli.py`, `io_handler.py`): parse arguments with argparse, accept `.jpg/.jpeg/.png/.bmp/.webp`, decode via `cv2.imread`, fail early with plain messages.
2. **Measurement** (`face_inspection`, `photo_stats`, `clarity`, `structure`, `palette`): five independent analysers, each a pure function of the pixel array plus explicit parameters. No shared state.
3. **Rendering** (`visualizer.py`, `reporter.py`): numbered boxes are drawn on a copy, images are written with auto-created folders, and both report formats are emitted together so text and JSON can never disagree.

6 Module Catalogue

Module	Job
`cli.py`	Argument parsing, stage orchestration, console recap.
`io_handler.py`	Suffix gate (`ALLOWED_TYPES`) and guarded OpenCV load.
`face_inspection.py`	Grayscale + Haar cascade face boxes (1.15 / 6 neighbours / 30 px min).
`photo_stats.py`	Resolution, megapixels, orientation, mean luma, contrast via `meanStdDev`.
`clarity.py`	Exposure label (cuts 55 / 195) and focus label (Laplacian cut 90).
`structure.py`	7×7 blur, Canny (60/140), edge density, external-contour count.
`palette.py`	Mean B/G/R intensity and dominant channel.
`visualizer.py`	Numbered boxes (`Face #i`) and safe image writing.
`reporter.py`	`summary.txt` + `summary.json` from one payload.

7 Methods and Parameters

7.1 Face Marking

The bundled frontal-face cascade scans the grayscale frame at scale factor 1.15 with six neighbours required and a 30 px minimum side. The cascade file ships inside `lensmetrics/models/` so the run does not depend on the OpenCV install layout. Each hit is drawn as a thick terracotta-blue box with a `Face #i` tag and its coordinates are preserved in the JSON record.

7.2 Geometry and Contrast

`cv2.meanStdDev` on the grayscale frame yields mean luma and its spread (contrast) in one call. Orientation follows directly from the width/height comparison; megapixels are the pixel count scaled to millions.

7.3 Exposure and Focus

Mean luma under 55 reads *Under-exposed*, over 195 *Over-exposed*, otherwise *Well-exposed*. Focus uses Laplacian variance with a cutoff of 90: at or above reads *In-focus*, below reads *Soft-focus*. Both cutoffs are constructor/CLI arguments, not buried constants. The reference photo scored luma 123.80 and blur 122.02 — comfortably in the well-exposed, in-focus band.

7.4 Structure Tracing

A 7×7 Gaussian softens sensor noise, Canny with hysteresis 60/140 keeps strong boundaries, and `findContours` (external mode) counts distinct regions. Edge density (active pixels ÷ total) complements the count: the reference frame carried 2.94% edge pixels across 859 contours, i.e. a busy but coherent outdoor scene.

7.5 Colour Balance

Mean intensity per B/G/R channel plus the argmax channel summarise cast at a glance. The reference frame measured B 115.44 / G 127.57 / R 119.59 — green dominant, consistent with foliage-heavy content.

8 Command Line and Failure Behaviour

```
pip install -r requirements.txt
python main.py --input input/image.jpeg --outdir output
python main.py -i input/image.jpeg -o output --canny-low 60 --canny-high 140 --
    blur-cut 90
pytest -q
```

Listing 1: Typical invocations

Failure	Signal	Response
Missing file	`FileNotFoundError`	CLI exits with the offending path quoted.
Bad suffix	`ValueError`	Allowed suffix list is printed.
Undecodable bytes	`ValueError`	“could not decode” naming the file.
Cascade missing	`RuntimeError`	Initialisation failure, no traceback spam.
Unwritable output	`IOError`	Target path named in the message.

9 Implementation Notes

Three excerpts show the codebase voice: small typed functions, dataclass results, explicit parameters.

```
ALLOWED_TYPES = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}

def resolve_source(path: str) -> str:
    if not os.path.isfile(path):
        raise FileNotFoundError(f"No photo found at: {path}")
    suffix = os.path.splitext(path)[1].lower()
    if suffix not in ALLOWED_TYPES:
        raise ValueError(f"File type '{suffix}' is not supported. ...")
    return path
```

Listing 2: Guarded intake in io_handler.py

```
def grade_clarity(bgr, dark_cut=55.0, bright_cut=195.0, blur_cut=90.0):
    grey = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)
    luma = float(grey.mean())
    blur = float(cv2.Laplacian(grey, cv2.CV_64F).var())
    exposure = ("Under-exposed" if luma < dark_cut
                else "Over-exposed" if luma > bright_cut
                else "Well-exposed")
    focus = "In-focus" if blur >= blur_cut else "Soft-focus"
    ...
```

Listing 3: Exposure and focus grading in clarity.py

```
soft = cv2.GaussianBlur(grey, (blur, blur), 0)
edge_map = cv2.Canny(soft, low, high)
contours, _ = cv2.findContours(edge_map, cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_SIMPLE)
```

Listing 4: Edge tracing with contour census in structure.py

10 Reference Run-Through

1. **Launch:** `python main.py -input input/image.jpeg -outdir output`.
2. **Intake:** suffix `.jpeg` accepted; 1200 × 1600 colour frame decoded.
3. **Faces:** cascade pass returns 2 boxes; both are tagged on a copy.
4. **Measures:** geometry (1.92 MP, portrait, contrast 66.04), clarity (123.80 / 122.02), structure (2.94% edges, 859 contours), palette (green dominant).
5. **Artefacts:** `annotated.jpg`, `edgemap.jpg`, `summary.txt`, `summary.json` land in `output/`.
6. **Recap:** the console prints faces, size, exposure, focus, and contour count for immediate reading.

11 Evidence



Figure 1: Reference input (`input/image.jpeg`), 1200 × 1600 portrait.



Figure 2: Face marking: 2 numbered boxes on the reference photo.

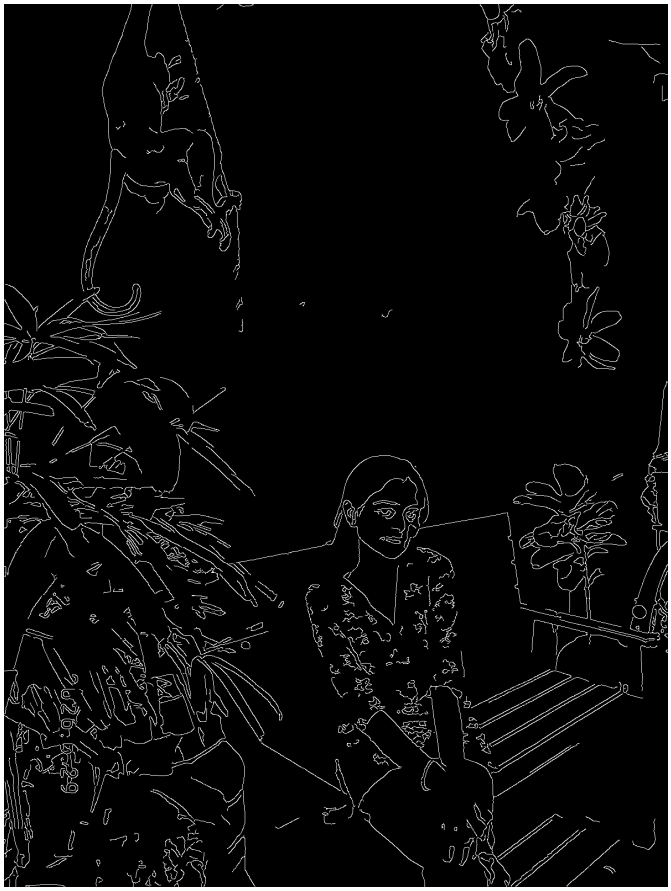


Figure 3: Structure map: Canny 60/140 after 7×7 blur (2.94% active).

```
===== LensMetrics =====
Photo      : input/image.jpeg
Faces      : 2
Size       : 1200 x 1600 (portrait)
Exposure   : Well-exposed
Focus      : In-focus
Contours   : 859
Saved to   : output/ (annotated.jpg, edgemap.jpg, summary.txt, summary.json)
```

Listing 5: Console recap of the reference run

```
LensMetrics Inspection Summary
==============================

Source photo : input/image.jpeg
Faces marked : 2

[ Geometry ]
Resolution   : 1200 x 1600 (portrait)
Channels     : 3
Megapixels   : 1.92
Mean luma    : 123.8
Contrast     : 66.04

[ Clarity ]
Exposure     : Well-exposed (123.8)
Focus        : In-focus (score 122.02)

[ Structure ]
```

```
Edge pixels   : 56365 (2.94%)
Contours      : 859

[ Colour ]
B/G/R means   : 115.44 / 127.57 / 119.59
Dominant      : green
```

Listing 6: Readable summary (output/summary.txt)

12 Testing

12.1 Automated Suite

Eight Pytest cases in `tests/test_lensmetrics.py` use synthetic frames (mid-grey, black, red-tinted) plus a temporary directory for report I/O:

```
tests/test_lensmetrics.py::test_geometry_and_orientation PASSED      [ 12%]
tests/test_lensmetrics.py::test_clarity_mid_grey_is_well_exposed PASSED [ 25%]
tests/test_lensmetrics.py::test_dark_frame_flags_under_exposed PASSED [ 37%]
tests/test_lensmetrics.py::test_structure_map_shape_and_census PASSED [ 50%]
tests/test_lensmetrics.py::test_palette_reports_dominant_channel PASSED [ 62%]
tests/test_lensmetrics.py::test_face_tagging_keeps_dimensions PASSED [ 75%]
tests/test_lensmetrics.py::test_reports_write_txt_and_json PASSED    [ 87%]
tests/test_lensmetrics.py::test_bad_paths_raise_helpful_errors PASSED [100%]

8 passed in 0.20s
```

Listing 7: Full suite result

12.2 Manual Checks

Check	Expectation	Observation	Verdict
Reference photo	4 artefacts written	annotated + edge map + txt + json present	PASS
Missing file	Path quoted in error	`No photo found at: ...`	PASS
`.txt` as input	Suffix error + allowed list	Clear `ValueError`	PASS
Exposure label	Well-exposed at 123.8	Matches band 55–195	PASS
Focus label	In-focus at 122.02	Above cutoff 90	PASS
Edge density	Ratio in [0,1]	0.0294 on reference	PASS
JSON schema	Tool + boxes + sections	All keys present	PASS

13 Difficulties and Fixes

1. **Cascade file vanished in OpenCV 5.** `cv2.data.haarcascades` no longer ships the XML, so the first run failed to initialise. *Fix:* vendor the cascade under `lensmetrics/models/` with a fallback to the install path — reproducible on any wheel.
2. **Cutoffs felt arbitrary.** Early brightness bands disagreed with test shots. *Fix:* re-tuned to 55/195 for exposure and 90 for focus against dark, mid, and bright frames, and exposed them as CLI flags.
3. **Even blur kernels crash Canny tuning.** An even kernel size silently misbehaves. *Fix:* coerce even values up by one inside `trace_structure`.
4. **Text and JSON drifted.** Two writers meant two truths. *Fix:* single `save_reports` builds both from one payload.

14 Takeaways, Limits, and Next Steps

14.1 Takeaways

Quantitative grades beat adjectives; dataclasses keep analysers honest and testable; vendoring data files beats assuming install layouts; one payload feeding two report formats prevents an entire class of bugs.

14.2 Limits

One photo per run; classical cascade misses profiled or occluded faces; hard-wired defaults suit general shots, not night or document scans; no GUI.

14.3 Next Steps

Batch-folder mode with a CSV index; optional DNN face path when weights are present; histogram PNGs beside the JSON; SSIM/noise metrics; a minimal drag-and-drop front end.

15 References

1. OpenCV team, *OpenCV Documentation*, https://docs.opencv.org/.
2. Viola, P. & Jones, M., "Rapid object detection using a boosted cascade of simple features," *CVPR*, 2001.
3. Canny, J., "A computational approach to edge detection," *IEEE TPAMI*, 8(6), 1986.
4. Bradski, G. & Kaehler, A., *Learning OpenCV*, O'Reilly, 2008.
5. NumPy developers, *NumPy Documentation*, https://numpy.org/doc/stable/.
6. pytest developers, *pytest Documentation*, https://docs.pytest.org/.
7. Python Software Foundation, *argparse Documentation*, https://docs.python.org/3/library/argparse.html.

16 Closing

LensMetrics turns an idle photo into a graded, illustrated record in under a second on CPU: 2 faces boxed and logged, exposure and focus labelled from measured cutoffs, 859 contours backing a 2.94% edge-density figure, and a colour profile naming green dominant — with four artefacts and eight green tests to prove it. The modular layout leaves clear seams for batch mode, learned detectors, and richer metrics next.